

Core OOP Concepts in Java

*Abstraction, Encapsulation, and
Inheritance*

Learning Objectives

By the end of this session, students will be able to:

1. **Differentiate** between Data Abstraction (concept) and Abstract Data Types (implementation).
2. **Implement** robust Encapsulation in Java using access modifiers and accessor methods.
3. **Construct** class hierarchies using Inheritance (extends) and understand constructor chaining.
4. **Analyze** the trade-offs of inheritance vs. composition in Java design.
5. **Apply** these three pillars to create maintainable, modular Java applications.

-

OOP in the Java Context

Java is fundamentally Object-Oriented

- Everything (except primitives) is an Object.
- Code is organized into **Classes** and **Packages**.
- The JVM (Java Virtual Machine) manages memory for these objects (Heap vs. Stack).

OOP in the Java Context

Java is fundamentally Object-Oriented

- Everything (except primitives) is an Object.
- Code is organized into **Classes** and **Packages**.
- The JVM (Java Virtual Machine) manages memory for these objects (Heap vs. Stack).

The Three Pillars We Cover Today:

1.Abstraction: Hiding complexity.

2.Encapsulation: Hiding data state.

3.Inheritance: Reusing behavior.

Note: While Polymorphism is crucial, it is a result of correctly implementing these three foundations.

Data Abstraction (Concept)

Definition:

The process of hiding complex implementation details and showing only the essential features of the object to the user.

- **Why use Abstraction?**
-  **Reduced Complexity:** Users don't need to know *how* a method works, only *what* it does.
-  **Security:** Internal logic is protected from external interference.
-  **Maintainability:** Implementation can change without affecting client code.
- **Real-world Analogy:**
Driving a car. You use the steering wheel and pedals (interface), but you don't need to know how the fuel injection system works (implementation).

Abstract Data Types (ADT) in Java

- **Definition:**

A mathematical model for data types where a type is defined by its behavior (operations) rather than its implementation.

```
// The ADT Specification (Interface)
List<String> names = new ArrayList<>();
// User interacts with the abstraction
names.add("Alice");
names.get(0);
// Implementation (ArrayList vs LinkedList) is hidden
// We could switch to new LinkedList<>() without changing client code
```

Key Characteristics of an ADT in Java:

Type Safety: Defined by Generics (<String>).

Operation Set: add, remove, get, size.

Implementation Independence: Client code depends on the List abstraction, not the ArrayList concrete class.

Encapsulation (The Mechanism)

Definition:

Bundling data (variables) and methods (functions) that operate on the data into a single unit (Class), and restricting direct access to some of an object's components.

Modifier	Class	Package	Subclass	World
Public	✓	✓	✓	✓
Protected	✓	✓	✓	✗
default (no keyword)	✓	✓	✗	✗
private	✓	✗	✗	✗

Encapsulation (The Mechanism)

Modifier	Class	Package	Subclass	World
Public	✓	✓	✓	✓
Protected	✓	✓	✓	✗
default (no keyword)	✓	✓	✗	✗
private	✓	✗	✗	✗

- **Golden Rule of Encapsulation:**



Fields should be private.



Access should be via public methods (Getters/Setters).

Encapsulation (Implementation Example)

Scenario: Protecting the state of a BankAccount.

```
public class BankAccount {  
    private String accountNumber; // 1. Private Data (Hidden)  
    private double balance;  
  
    // 2. Public Constructor  
    public BankAccount(String accountNumber, double initialBalance) {  
        this.accountNumber = accountNumber;  
        this.balance = initialBalance; // Validation could go here  
    }  
    // 3. Controlled Access (Getter)  
    public double getBalance() {  
        return balance; // Read-only access  
    }  
    // 4. Controlled Modification (Setter/Method)  
    public void deposit(double amount) {  
        if (amount > 0) {  
            this.balance += amount;  
        } else {  
            throw new IllegalArgumentException("Amount must be positive");  
        }  
    }  
}
```

Inheritance (Basics)

Definition:

A mechanism where one class acquires the properties (fields) and behaviors (methods) of another class.

Key Keywords:

- extends: Used for class inheritance.
- super: Refers to the immediate parent class object.
- Object: The root of all Java class hierarchies.

The "Is-A" Relationship:

Inheritance should only be used when there is a clear **IS-A** relationship.

-  Dog **is a** Animal.
-  Car **is a** Engine (Wrong! A Car *has an* Engine → Use Composition).

Inheritance (Basics)

```
public class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

```
public class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking...");  
    }  
}
```

Inheritance & Constructors

Concept:

Constructors are **not** inherited, but the parent constructor **must** be called.

Constructor Chaining:

- If a subclass constructor does not explicitly call `super()`, Java inserts a no-arg `super()` call automatically.
- If the parent class lacks a no-arg constructor, the subclass **must** explicitly call a parent constructor.

Inheritance & Constructors

```
public class Person {  
    protected String name;  
    public Person(String name) {  
        this.name = name;  
    }  
}
```

```
public class Employee extends Person {  
    private int employeeId;  
  
    public Employee(String name, int id) {  
        super(name); // MUST call parent constructor first  
        this.employeeId = id;  
    }  
}
```

Order of Initialization:

Static blocks → Parent Constructor → Child Constructor.

Class Hierarchy Design

The Java Class Tree:

- Every class implicitly extends `java.lang.Object` if no other class is specified.
- **Single Inheritance:** A Java class can extend only **one** direct parent class.
- **Multilevel Inheritance:** Class C extends B, Class B extends A.

Designing Hierarchies:

- **Generalization:** Moving common code up to the parent (Superclass).
- **Specialization:** Adding specific features in the child (Subclass).
- **Depth vs. Width:** Prefer wider hierarchies over deep ones (Deep trees are fragile).

Class Hierarchy Design

Visualizing the Hierarchy:

Object



Animal (Abstract/Common behavior)



Dog (Specific behavior)

Integrated Case Study: University System

Combining Abstraction, Encapsulation, and Inheritance

```
public class Person {
    private String name; // Encapsulated
    public Person(String name) { this.name = name; }
    public String getName() { return name; }
}

public class Student extends Person {
    private int studentId; // Specific data

    public Student(String name, int id) {
        super(name); // Constructor Chaining
        this.studentId = id;
    }

    public void enrollCourse(String course) {
        System.out.println(getName() + " enrolled in " + course);
    }
}

List<Person> campusMembers = new ArrayList<>();
campusMembers.add(new Student("Alice", 101));
```

Common Pitfalls in Java OOP

1. Breaking Encapsulation:

- ✗ public int balance; (Allows invalid state)
- ✓ private int balance; + public void deposit()

2. Inheritance Abuse:

- ✗ class Stack extends Vector (Stack is not a Vector)
- ✓ class Stack { private Vector data; } (Composition)

3. Fragile Base Class Problem:

- Changing logic in the Parent class can unintentionally break Child classes.
- **Solution:** Favor composition over inheritance when possible.

4. God Classes:

- A class that knows too much or does too much (violates abstraction).
- **Solution:** Split into smaller, focused classes.

Summary & Key Takeaways

- Abstraction focuses on what an object does; ADTs (like Java Collections) allow swapping implementations without breaking code.
- Encapsulation protects object integrity using private fields and public methods with validation.
- Inheritance promotes code reuse via extends but requires careful design of constructors (super) and hierarchies.
- Design Rule: Use inheritance for "Is-A" relationships; use composition for "Has-A" relationships.
- Java Specific: All classes inherit from Object; single class inheritance is enforced.

Q&A and Discussion

? Question: was inheritance used incorrectly?

Square extends Rectangle ...

? Question: When should I use protected vs private in a parent class?

? Question: Why does Java not support multiple class inheritance?

💬 Discussion: Share a time when changing a parent class broke your child classes.